CODYSSEY 1-1 · 통합 검증 및 심층 인터뷰 마스터북

🛠️ 내 컴퓨터에 개발자용 '작업실' 꾸미기

리눅스 CLI · 파일 권한(chmod) · Docker 이미지/컨테이너/포트/볼륨 · Git/GitHub 연동 — 평가문항 1~5 전체 통합본

Host & Container OS	Docker Engine	Git & GitHub	최종 평가 결과
macOS 26.6.2 / Ubuntu 22.04	v29.2.1 (LinuxKit 6.12.72)	git 2.50.1 / gamercross	전체 PASS (18/18) + 보너스 부여

📋 평가문항 전체 체크리스트 요약표 (항목 1 ~ 항목 5)

항목 1. 기능 동작 검증 (9개 문항) — 결과: PASS

[x] **1-1.** 터미널에서 기본 명령어로 폴더/파일 생성·이동·삭제를 수행한 흔적이 있는가? (`mkdir, touch, cp, mv, rm, ls -la`)

[x] **1-2.** 파일 권한 변경 결과가 확인되는가? (`chmod 644 hello.sh` 실행 거부 ➜ `chmod 755 hello.sh` 실행 성공)

[x] **1-3.** `docker --version`이 출력되고, Docker가 동작 가능한 상태인가? (`Docker 29.2.1 / Server & Kernel 정상`)

[x] **1-4.** `docker run hello-world`가 정상 실행되는가? (`Hello from Docker!` 및 `Exited (0)` 확인)

[x] **1-5.** 이미지/컨테이너 목록 확인 및 정리 흔적이 있는가? (`docker ps -a, images, rm → rmi` 순서 및 충돌 에러 검증)

[x] **1-6.** `Dockerfile`로 이미지 빌드가 가능한가? (`docker build -t codyssey-web:1.0 -f Dockerfile .` 빌드 완료)

[x] **1-7.** 매핑된 포트로 접속이 가능한가? (`-p 8080:80`, `-p 8081:80` 실행 후 `curl 200 OK` 및 브라우저 캡처)

[x] **1-8.** Docker 볼륨 데이터가 컨테이너 삭제 후에도 유지되는가? (`codyssey-data` 볼륨 + `docker rm -f` 전/후 데이터 생존 확인)

[x] **1-9.** Git 설정 및 GitHub 연동이 확인되는가? (`git config, git remote -v, gh auth status` 마스킹 및 원격 푸시 완료)

항목 2. 동작 구조 설계 (2개 문항) — 결과: PASS

[x] **2-1.** 프로젝트 디렉토리 구조를 어떤 기준으로 구성했는지 설명할 수 있는가? (역할·변경 빈도·빌드 캐시 격리 기준)

[x] **2-2.** 포트/볼륨 설정을 어떤 방식으로 재현 가능하게 정리했는지 설명할 수 있는가? (CLI 표준 명령어 + `docker-compose.yml` 코드 고정)

항목 3. 핵심 기술 원리 적용 (4개 문항) — 결과: PASS

[x] **3-1.** 이미지와 컨테이너의 차이를 "빌드/실행/변경" 관점에서 구분해 설명할 수 있는가? (불변 RO 스냅샷 vs 가변 RW 프로세스)

[x] **3-2.** 컨테이너 내부 포트로 직접 접속할 수 없는 이유와 포트 매핑이 필요한 이유를 설명할 수 있는가? (네트워크 네임스페이스 격리와 DNAT 포워딩)

[x] **3-3.** 절대 경로/상대 경로를 어떤 상황에서 선택하는지 설명할 수 있는가? (컨테이너 내부 고정 위치 vs 프로젝트 이식성)

[x] **3-4.** 파일 권한 숫자 표기(예: `755`, `644`)가 어떤 규칙으로 결정되는지 설명할 수 있는가? (`r=4, w=2, x=1` 합산 및 파일/폴더 `x` 차이)

항목 4. 심층 인터뷰 (3개 문항) & 항목 5. 보너스 문제 해결 — 결과: PASS / 크레딧: 부여

[x] **4-1.** "호스트 포트가 이미 사용 중"이라 포트 매핑이 실패한다면, 어떤 순서로 원인을 진단할지 설명할 수 있는가?

[x] **4-2.** 컨테이너 삭제 후 데이터가 사라진 경험이 있다면, 이를 방지하기 위한 대안을 설명할 수 있는가?

[x] **4-3.** 이 미션에서 가장 어려웠던 지점과 해결 과정(`가설 → 확인 → 조치`)을 근거와 함께 설명할 수 있는가?

[x] **항목 5 (보너스).** `docker-compose.yml` 선언형 서비스 구성 및 `up -d / ps / curl / down` 운영 검증 완료

항목 1. 기능 동작 검증 (1-1 ~ 1-9 상세 실행 흔적)

결과: PASS

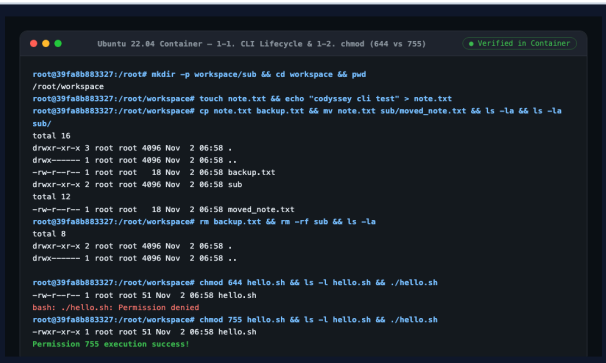
[x] 1-1. 터미널 기본 명령어로 폴더/파일 생성·이동·삭제 & [x] 1-2. 파일 권한(644 vs 755) 변경 검증

> 💡 **비전공자 눈높이 비유:** 일반 안내문 파일은 주인만 고치고 남들은 눈으로 읽기만 하라고 `644(-rw-r--r--)`를 주고, 커피머신 작동 버튼처럼 눌러서 실행해야 하는 스크립트(`hello.sh`)에는 실행 점수(`x=1`)를 더한 `755(-rwxr-xr-x)`를 줍니다.

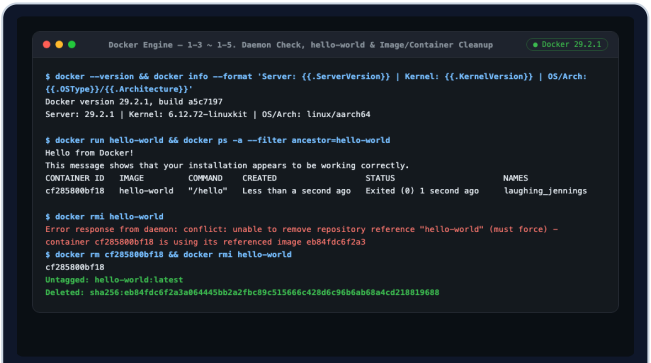
```
# [1-1] ubuntu:22.04 컨테이너 내부에서 폴더/파일 생성 → 복사 → 이동/이름변경 → 삭제
$ mkdir -p workspace/sub && cd workspace && pwd
/root/workspace
$ touch note.txt && echo "codyssey cli test" > note.txt
$ cp note.txt backup.txt && mv note.txt sub/moved_note.txt && ls -la && ls -la sub/
drwxr-xr-x 3 root root 4096 Nov  2 06:58 .
-rw-r--r-- 1 root root   18 Nov  2 06:58 backup.txt
drwxr-xr-x 2 root root 4096 Nov  2 06:58 sub
-rw-r--r-- 1 root root   18 Nov  2 06:58 moved_note.txt
$ rm backup.txt && rm -rf sub && ls -la
total 8 (backup.txt 및 sub/ 폴더 완전 삭제 확인)

# [1-2] 파일 권한 변경 전(644: 실행 거부) vs 변경 후(755: 실행 성공)
$ chmod 644 hello.sh && ls -l hello.sh
-rw-r--r-- 1 root root 51 Nov  2 06:58 hello.sh   # [chmod 644 직후: 실행 비트(x) 없음]
$ ./hello.sh
bash: ./hello.sh: Permission denied

$ chmod 755 hello.sh && ls -l hello.sh
-rwxr-xr-x 1 root root 51 Nov  2 06:58 hello.sh   # [chmod 755 직후: 소유자/그룹/기타 모두 x 부여]
$ ./hello.sh
Permission 755 execution success!
```



▲ 1-1 & 1-2. 컨테이너 내부 CLI 및 chmod 644/755 실행 캡처



▲ 1-3 ~ 1-5. Docker 버전·데몬·hello-world 및 rm/rmi 정리 캡처

[x] 1-3. Docker 버전/데몬 확인 · [x] 1-4. hello-world 실행 · [x] 1-5. 컨테이너/이미지 목록 및 정리 흔적

```
# [1-3] Docker CLI 버전 및 Server/Kernel 데몬 상태 점검
$ docker --version && docker info --format 'Server: {{.ServerVersion}} | Kernel: {{.KernelVersion}} |
OS/Arch: {{.OSType}}/{{.Architecture}} | Containers: {{.Containers}} | Images: {{.Images}}'
Docker version 29.2.1, build a5c7197
Server: 29.2.1 | Kernel: 6.12.72-linuxkit | OS/Arch: linux/aarch64 | Containers: 0 | Images: 5

# [1-4] hello-world 실행 및 직후 컨테이너 상태(Exited 0) 스냅샷
$ docker run hello-world && docker ps -a --filter ancestor=hello-world
Hello from Docker! This message shows that your installation appears to be working correctly.
CONTAINER ID   IMAGE         COMMAND    CREATED                  STATUS
NAMES
cf285800bf18   hello-world   "/hello"   Less than a second ago   Exited (0) Less than a second ago
laughing_jennings

# [1-5] 컨테이너가 남아 있을 때 이미지 삭제(rmi) 충돌 확인 후 rm → rmi 순서로 정리
$ docker rmi hello-world
Error response from daemon: conflict: unable to remove repository reference "hello-world" (must forc
e) - container cf285800bf18 is using its referenced image eb84fdc6f2a3
$ docker rm cf285800bf18 && docker rmi hello-world
cf285800bf18
Untagged: hello-world:latest / Deleted: sha256:eb84fdc6f2a3a064445bb2a2fbc89c515666c428d6c96b6ab68a4c
d218819688
```

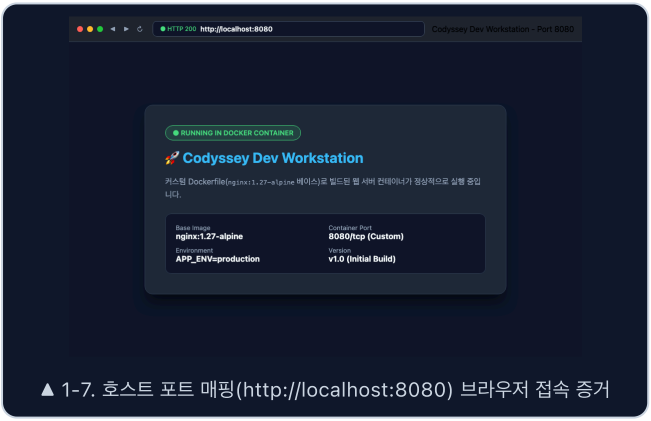
[x] 1-6. Dockerfile 커스텀 이미지 빌드 · [x] 1-7. 포트 매핑 접속 · [x] 1-8. 볼륨 영속성 · [x] 1-9. Git/GitHub 연동

```
# [1-6] Dockerfile로 커스텀 웹 서버 이미지(codyssey-web:1.0) 빌드 (-t: 태그, -f: 파일, .: 빌드 컨텍스트)
$ docker build -t codyssey-web:1.0 -f Dockerfile . && docker images codyssey-web:1.0
[+] Building 2.0s (9/9) FINISHED  => naming to docker.io/library/codyssey-web:1.0
IMAGE              ID             DISK USAGE
codyssey-web:1.0   bbcb31fc5683       49.7MB

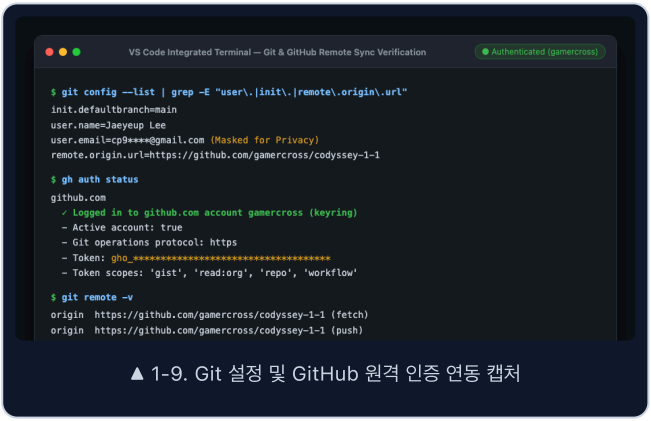
# [1-7] 호스트 포트 매핑(-p 8080:80, -p 8081:80) 실행 및 HTTP 200 OK 응답 확인
$ docker run -d --name codyssey-app -p 8080:80 codyssey-web:1.0 && curl -I http://localhost:8080
HTTP/1.1 200 OK (Server: nginx/1.27.5, 0.0.0.0:8080->80/tcp)

# [1-8] Named Volume(codyssey-data) 생성·점검 및 컨테이너 강제 삭제(docker rm -f) 후 데이터 생존 증명
$ docker volume create codyssey-data && docker volume inspect codyssey-data --format '{{.Name}}: {{.M
ountpoint}}'
codyssey-data: /var/lib/docker/volumes/codyssey-data/_data
$ docker run -d --name vol-test-1 -p 8080:80 -v codyssey-data:/usr/share/nginx/html/data codyssey-we
b:1.0
$ docker exec vol-test-1 sh -c 'echo "Persistent Data Across Container Deletion" > /usr/share/nginx/h
tml/data/persist.txt'
$ docker rm -f vol-test-1
$ docker run -d --name vol-test-2 -p 8080:80 -v codyssey-data:/usr/share/nginx/html/data codyssey-we
b:1.0
$ curl http://localhost:8080/data/persist.txt
Persistent Data Across Container Deletion

# [1-9] Git 설정 및 GitHub 원격 저장소 인증 확인 (토큰 마스킹)
$ git config --get user.name && git remote -v && gh auth status
Jaeyeup Lee / origin  https://github.com/gamercross/codyssey-1-1 (fetch & push)
✓ Logged in to github.com account gamercross (Token: gho_************************************)
```



▲ 1-7. 호스트 포트 매핑(http://localhost:8080) 브라우저 접속 증거



▲ 1-9. Git 설정 및 GitHub 원격 인증 연동 캡처

항목 2. 동작 구조 설계 (디렉토리 분리 기준 & 포트/볼륨 재현성) 결과: PASS

[x] 2-1. 프로젝트 디렉토리 구조를 어떤 기준으로 구성했는지 설명할 수 있는가?

설계 핵심 기준: 파일의 **역할(인프라 명세 / 웹 애플리케이션 코드 / 검증·원리 문서)**과 **변경 빈도**를 기준으로 분리하여, 문서(`docs/`, `README.md`)를 수정하더라도 Docker 이미지 빌드 시 `COPY app/` 레이어 캐시가 무효화되지 않도록 설계했습니다.

구분 영역	폴더 및 파일	분리 기준 및 설계 이유
인프라 명세 (루트)	`Dockerfile`, `docker-compose.yml`	저장소를 클론한 사람이 루트 디렉토리에서 즉시 이미지 빌드(`docker build .`)와 원클릭 기동(`docker compose up -d`)을 수행할 수 있도록 최상위 진입점에 배치
실행 대상 (`app/`)	`app/index.html`	컨테이너 내부(`/usr/share/nginx/html/`)로 복사할 정적 웹 자산만 별도 폴더로 격리하여 불필요한 파일 유입 차단 및 레이어 캐시 효율 극대화
검증·원리 문서 (`docs/`)	`01-verification-log.md`, `02-core-principles.md`, `03-deep-interview-qa.md`, `images/`	대문(`README.md`)은 전체 설계와 체크리스트 라우터 역할을 맡고, 상세 실행 로그(How)·기술 원리(Why)·트러블슈팅(QA)을 모듈별로 분리해 추적성 확보

[x] 2-2. 포트/볼륨 설정을 어떤 방식으로 재현 가능하게 정리했는지 설명할 수 있는가?

사람 기억에 의존해 매번 긴 `docker run` 옵션을 손으로 입력하면 포트 번호 오타나 볼륨 누락으로 인한 데이터 유실이 발생합니다. 이를 막기 위해 **① 고정된 표준 CLI 명령어**와 **② 선언형 설정 파일(`docker-compose.yml`) 코드화**의 2단계로 재현성을 확보했습니다.

```yaml
# [docker-compose.yml 전문 — 포트(8080:80)와 영속 볼륨(codyssey-data)을 코드로 고정]
name: codyssey-1-1
services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    image: codyssey-web:1.0
    container_name: codyssey-app
    ports:
      - "8080:80"
    volumes:
      - codyssey-data:/usr/share/nginx/html/data
    restart: unless-stopped
volumes:
  codyssey-data:
    name: codyssey-data
```

> **호스트 OS별 재현 주의사항:** macOS/Windows(OrbStack·Docker Desktop)는 경량 Linux VM 위에서 동작하므로 `sudo` 없이 실행 가능하며 VM 프록시가 포트를 포워딩합니다. 반면 Linux Native 환경에서는 `docker` 그룹 권한(또는 `sudo`)이 필요하며, `-p 8080:80` 사용 시 호스트 방화벽을 우회해 외부(`0.0.0.0`)에 노출되므로 로컬 전용 테스트 시에는 `-p 127.0.0.1:8080:80` 바인딩을 권장합니다.

항목 3. 핵심 기술 원리 적용 (4대 원리 비교 및 비전공자 눈높이 해설) 결과: PASS

[x] 3-1. 이미지와 컨테이너의 차이를 "빌드 / 실행 / 변경" 관점에서 구분해 설명할 수 있는가?

> 🥯 **비전공자 비유 (붕어빵 틀 vs 붕어빵): 이미지**는 한 번 주조하면 모양이 바뀌지 않는 **'붕어빵 틀(설계도)'**이고, **컨테이너**는 그 틀로 찍어내어 접시 위에 올려둔 **'붕어빵(실행 프로세스)'**입니다. 붕어빵 하나를 먹어치워도(컨테이너 삭제) 틀(이미지)은 그대로 남아 언제든 새 붕어빵을 똑같이 찍어낼 수 있습니다.

구분 관점	Docker 이미지 (Image)	Docker 컨테이너 (Container)
1. 빌드 (Build)	`Dockerfile` 지시어(`FROM, WORKDIR, COPY, RUN`)를 적층해 만든 **읽기 전용(Read-Only) 스냅샷**	빌드 대상이 아니며, 이미 빌드된 이미지 스냅샷을 기반으로 1초 만에 생성됨
2. 실행 (Run)	디스크에 저장된 정적 파일 묶음 (CPU·메모리 미사용)	이미지 위에 **얇은 읽기/쓰기(Read-Write) 레이어**와 격리된 네트워크·PID를 할당받아 돌아가는 **실제 프로세스**
3. 변경 (Mutate)	**불변(Immutable):** 내용을 영구 변경하려면 호스트 코드 수정 후 `docker build -t codyssey-web:1.1 .`로 새로 빌드해야 함	**가변·휘발성(Ephemeral):** `docker exec`로 내부 파일을 고쳐도 컨테이너 삭제(`docker rm`) 시 RW 레이어와 함께 사라짐

[x] 3-2. 컨테이너 내부 포트로 직접 접속할 수 없는 이유와 포트 매핑이 필요한 이유를 설명할 수 있는가?

> 🚪 **비전공자 비유 (방음 부스와 대문 인터폰):** 컨테이너는 집 안에 설치한 **'완벽한 방음 부스'**라 밖에서 바로 문을 두드릴 수 없습니다. 그래서 집 대문(호스트 `8080번`)에 초인종을 달고, 그걸 누르면 방음 부스 안의 전화기(컨테이너 `80번`)로 연결되게 전선을 이어준 것이 **포트 매핑(`-p 8080:80`)**입니다.

- **직접 접속 불가 이유:** 컨테이너는 실행될 때 호스트와 격리된 독립 **네트워크 네임스페이스(Network Namespace)**와 가상 브리지 사설 IP를 할당받기 때문입니다. (`Dockerfile`의 `EXPOSE 80`은 문서화 표시일 뿐 실제 포트를 열지 않음)
- **포트 매핑의 역할 및 보안:** 호스트의 `8080` 포트로 들어온 패킷을 컨테이너의 `80` 포트로 포워딩(DNAT)해 줍니다. 이때 `-p 8080:80`은 `0.0.0.0:8080`(외부 전체 공개)으로 열리므로, 로컬 전용 접근만 허용할 때는 `-p 127.0.0.1:8080:80`으로 바인딩 범위를 제한합니다.

[x] 3-3. 절대 경로/상대 경로 선택 기준 & [x] 3-4. 파일 권한 숫자 표기(755, 644) 결정 규칙

3-3. 절대 경로(`/...`) vs 상대 경로(`./...`) 선택 기준	3-4. 파일 권한 숫자 표기(`755`, `644`) 결정 규칙
• **절대 경로 (도로명 전체 주소):** 루트(`/`)부터 시작. 현재 위치(`pwd`)와 무관하게 **컨테이너 내부의 고정된 시스템 경로**를 지정할 때 선택 (예: `WORKDIR /usr/share/nginx/html`) • **상대 경로 ("내 바로 옆방"):** 현재 위치(`.`) 기준. 프로젝트 폴더를 **다른 PC로 클론해도 깨지지 않는 이식성**이 필요할 때 선택 (예: `COPY app/ ./`, 마크다운 상대 링크)	• **점수 합산 규칙: 소유자 · 그룹 · 기타** 3자리에 각각 `r(읽기=4) + w(쓰기=2) + x(실행=1)` 값을 합산 • **`644 (-rw-r--r--)`**: `6(4+2)/4/4` — 일반 문서·설정 파일 (실행 불가) • **`755 (-rwxr-xr-x)`**: `7(4+2+1)/5(4+1)/5(4+1)` — 실행 스크립트 및 디렉토리 • **파일 vs 폴더 `x(1)` 차이:** 파일의 `x`는 프로세스 실행 권한이고, 디렉토리의 `x`는 `cd`로 폴더 내부에 진입할 수 있는 탐색 권한임!

항목 4. 심층 인터뷰 (3문항 실험 근거 & PREP 구술 카드) & 항목 5. 보너스 결과: PASS / 보너스: 부여

[x] 4-1. "호스트 포트가 이미 사용 중"이라 포트 매핑이 실패할 때의 진단 순서

```
# [일부러 실패해 본 실험: 8080 포트가 이미 점유된 상태에서 중복 바인딩 시도]
$ docker run -d --name codyssey-app-conflict -p 8080:80 codyssey-web:1.0
docker: Error response from daemon: ... Bind for 0.0.0.0:8080 failed: port is already allocated

# [진단 1단계: Docker 컨테이너 중 8080 포트 점유 주체 확인]
$ docker ps --filter publish=8080 --format "table {{.ID}}\t{{.Names}}\t{{.Ports}}"
6fd140c60216   codyssey-app   0.0.0.0:8080->80/tcp

# [진단 2단계: 호스트 OS 레벨에서 8080 포트 점유 프로세스(PID) 확인]
$ lsof -i :8080 | head -n 2
COMMAND     PID    USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
com.docke 65420 <USER>  152u  IPv6 0x18dbffa2a10556e5      0t0  TCP *:http-alt (LISTEN)

# [조치: 기존 서비스 유지 시 호스트 매핑 포트만 8081로 분리하여 동시 구동 성공!]
$ docker rm -f codyssey-app-conflict && docker run -d --name codyssey-app-8081 -p 8081:80 codyssey-we
b:1.0
```

🎙️ **60초 구술 답변 (PREP):**

- **P(결론):** 포트 충돌 시 ① **에러 로그로 포트 번호 특정** → ② **`docker ps --filter publish=포트`로 컨테이너 확인** → ③ **`lsof -i :포트`로 호스트 PID 확인** → ④ **중지하거나 호스트 매핑 포트만 변경**하는 4단계로 진단합니다.
- **R(이유):** 하나의 호스트 포트에는 하나의 소켓만 바인딩할 수 있는데, 원인이 기존 Docker 컨테이너일 수도 있고 호스트 로컬 프로세스일 수도 있기 때문입니다.
- **E(사례) & 유의점:** 실험에서 `8080` 충돌을 재현한 뒤 `docker ps`와 `lsof`로 `codyssey-app`을 특정했고, 컨테이너 내부 포트(`80`)는 그대로 둔 채 호스트 포트만 `-p 8081:80`으로 분리해 두 서버를 동시에 띄웠습니다. (※ 컨테이너 점유 포트는 `com.docker.backend`를 `kill -9`하지 말고 반드시 `docker stop`으로 내리며, 일반 프로세스는 `kill -15` 정상 종료를 우선 시도합니다.)

[x] 4-2. 컨테이너 삭제 후 데이터가 사라진 경험과 이를 방지하기 위한 대안

```
# [실험 A: 볼륨 없이 컨테이너 RW 레이어에 파일 생성 후 삭제 → 유실 발생]
$ docker run -d --name ephemeral-test -p 8080:80 codyssey-web:1.0
$ docker exec ephemeral-test sh -c 'echo "Temp" > /usr/share/nginx/html/temp.txt' && docker rm -f eph
emeral-test
$ docker run -d --name ephemeral-test-2 -p 8080:80 codyssey-web:1.0
$ docker exec ephemeral-test-2 cat /usr/share/nginx/html/temp.txt
cat: can't open '/usr/share/nginx/html/temp.txt': No such file or directory

# [실험 B: Named Volume(codyssey-data) 연결 시 컨테이너 삭제 후에도 보존 + tar 백업 절차]
$ docker run --rm -v codyssey-data:/data:ro -v "$(pwd):/backup" ubuntu:22.04 tar czf /backup/backup.t
ar.gz -C /data .
```

🎙️ **60초 구술 답변 (PREP):**

- **P(결론):** 컨테이너 삭제 시 데이터 유실을 막기 위해, 데이터 생명주기를 컨테이너와 분리하는 **Named Volume(운영 데이터 영속화)**과 **Bind Mount(개발 중 코드 실시간 반영)**를 목적에 맞게 구분해 사용합니다.
- **R(이유):** 컨테이너의 기본 RW 레이어는 컨테이너에 종속된 임시 공간(일회용 팝업스토어)이라 `docker rm` 시 함께 소멸하기 때문입니다.
- **E(사례):** 볼륨 없이 만든 `temp.txt`는 컨테이너 삭제 후 사라졌지만, 외부 금고 격인 `codyssey-data` 볼륨에 저장한 `persist.txt`는 `vol-test-1`을 강제 삭제하고 `vol-test-2`를 새로 띄워도 그대로 보존되었습니다.

[x] 4-3. 가장 어려웠던 지점과 해결 과정 (`가설 → 확인 → 조치`) & [x] 항목 5. 보너스 과제

```
# [문제 상황 (Symptom): 한글 폴더('미션-수행')에서 보너스 과제 docker compose 실행 시 에러 발생]
$ basename "$(pwd)" && docker compose up -d
미션-수행
project name must not be empty (Exit Code: 14)

# [원인 가설 (Hypothesis) & 확인 (Verification)]
# 가설: Compose는 현재 폴더명에서 영문 소문자·숫자만 남겨 프로젝트 이름을 짓는데, 순수 한글 폴더명이라 빈 문자열("")이 되었을 것!
$ docker compose -p codyssey-1-1 config --services
web  # <-- 영문 프로젝트 이름을 주입하자 정상 파싱 확인!

# [근본 조치 (Action): docker-compose.yml 최상단에 'name: codyssey-1-1'을 코드로 영구 고정]
$ docker compose up -d && docker compose ps && docker compose down
✔ Network codyssey-1-1_default Created / ✔ Container codyssey-app Started (0.0.0.0:8080->80/tcp)
```

60초 구술 답변 (PREP):

- **P(결론):** 한글 작업 폴더(`미션-수행`)에서 `docker compose up -d` 실행 시 발생한 `project name must not be empty` 에러를 **`가설 → 확인 → 조치`** 프로세스로 분석해 어느 폴더에서도 깨지지 않는 구조로 해결했습니다.
- **R(이유):** 단순히 폴더 이름을 바꾸거나 매번 `-p` 옵션을 수동 입력해 우회하면, 다른 평가자가 한글 경로에 클론했을 때 동일한 장애가 재발하기 때문입니다.
- **E(사례):** 디렉토리명 정규화 규칙 때문에 한글이 제거되어 빈 문자열이 되었다는 가설을 세워 `-p codyssey-1-1`로 확인한 뒤, `docker-compose.yml` 최상단에 `name: codyssey-1-1`을 명시해 원클릭 재현성을 완성했습니다.